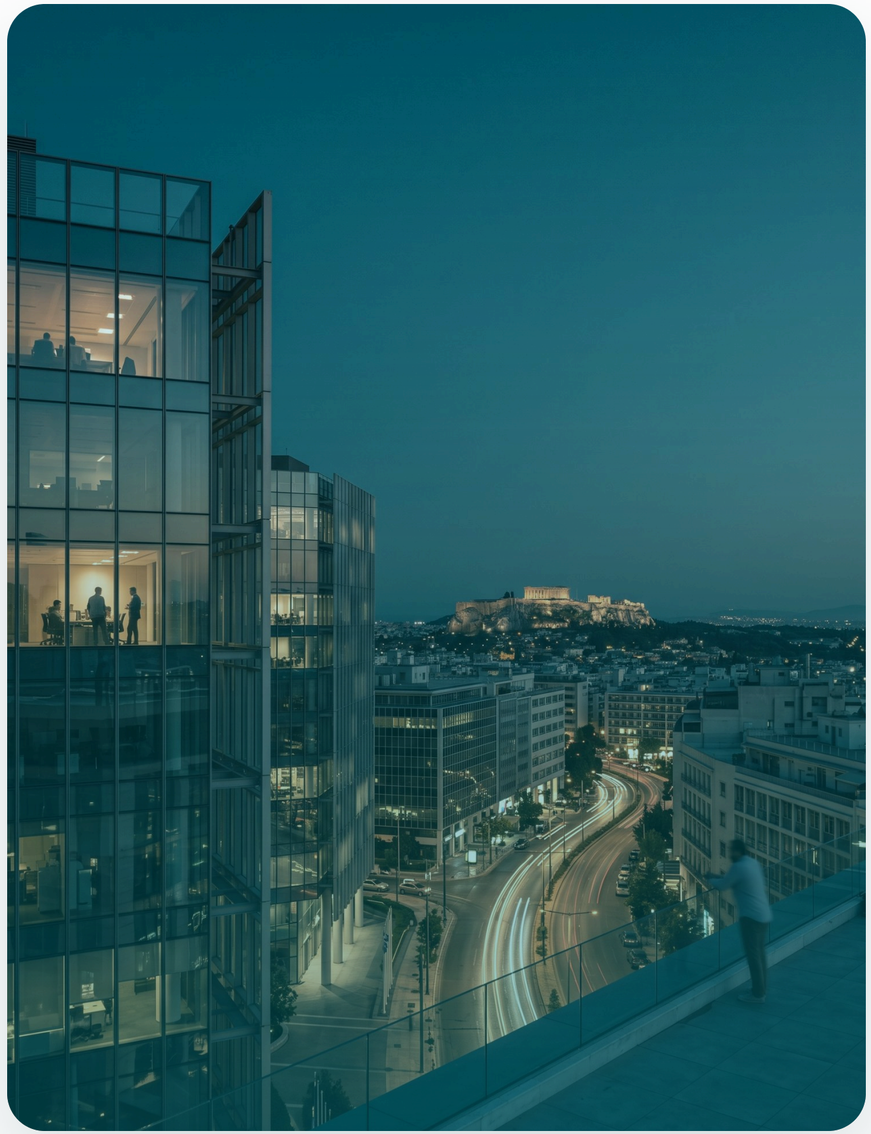
A PRESENTATIONS SKILL INSPIRED BY THE NBG BRAND

The nbg-design skill

How it is built, how it works, and how anyone who receives a deck can edit it.

Skill v1.25.0 • Oct 2026

NATIONAL BANK OF GREECE



NBG-DESIGN · OCT 2026

Agenda

Three sections. Each stands on its own.

- 01 WHAT IT MAKES POSSIBLE
- 02 ASK, BUILD, EDIT
- 03 MAKE IT YOUR OWN

01

SECTION 01

What it makes possible

A design system inspired by the NBG brand, a build pipeline and an in-deck editor, packaged as one Claude skill.

NATIONAL BANK OF GREECE

WHAT THE SKILL IS

One Claude Code skill that turns a request into a finished presentation in a look inspired by the NBG brand.

A design system in a box

- A design system **inspired by the NBG brand**: 16:9 artboard, Aptos, a nine-colour teal-led palette.
- Nine slide templates (covers, dividers, content), logos and photography bundled with the skill.
- Guardrails written down once, applied to every deck.

A deterministic build pipeline

- Claude authors the HTML; zero-dependency scripts embed the images, add the editor, verify, write the rebuild script, screenshot and export.
- A strict, browser-free gate that a deck must pass before it is delivered.
- The PDF and the PowerPoint file are made from the deck itself, so they look like the HTML.

Decks the recipients can edit

- Every delivered deck carries a right-click editor: fix a word, resize a card, align a row, ask an AI assistant.
- Nothing to install: it works from a file on the desktop, in any modern browser.
- Edits stay inside the look: the skill's fonts and palette only.

1 file
the whole deck: HTML with every image embedded

0
installs for the viewer, npm packages for the builder

HTML · PDF · PPTX
plus a rebuild script that refreshes the editor later; the PowerPoint file is editable, never screenshots

en · gr · bi
languages supported by the design system

WHY IT MATTERS

Four problems every presentation team knows, and how the skill answers each.

Consistency

Every deck shares one look. One palette, one type stack, one artboard, one logo lockup — enforced by the skill, not by the author's memory.
The verifier rejects a deck with a fake logo, a missing photo or an external image path.

Speed

From any content to a finished deck in minutes — a brief in chat, notes, a document, a spreadsheet, an existing deck or a Google Slides draft. Once the generator exists, every re-run is one command.

Portability

One self-contained file. It can be emailed, put on a share or opened from a USB stick; it renders identically everywhere because nothing points back to the machine that built it.

Ownership

Recipients fix their own slides. A typo, a longer title, a card that should be teal, a row that needs aligning — done in place, saved as a copy, exported to PDF — without a regeneration.

KEY POINT

The skill moves the effort from **making slides look right** to **deciding what they should say**.

WHAT YOU RECEIVE

The files of a deck, produced and verified the same way every time.

THE DELIVERABLES

- **The HTML deck** — 1920×1080 slides that scale to any screen, keyboard and button navigation, every logo and photo embedded, the right-click editor inlined.
- **The PDF** — printed from that HTML by the browser engine: one page per slide, vector text, same colours and photography, verified page count.
- **The PowerPoint file** (on request) — converted from that HTML element by element: text boxes, shapes and pictures you edit in PowerPoint, checked slide by slide against the deck.
- **The rebuild script** — `deck.rebuild.mjs`: run it after the skill is updated and the deck receives the newer editing tools, verified, with its PDF re-exported — no regeneration.
- **A generator** for data-driven decks — a small Python script that rebuilds the deck from its content, so updates are a diff, not a redo.

WHAT THE SKILL ASKS BEFORE IT STARTS

- **Topic, audience, slide count.** The three inputs it never guesses; it asks instead.
- **Language** — English by default; Greek or bilingual on request.
- **Output** — HTML by default; "send as PDF" adds the PDF, "in PowerPoint" an editable .pptx.
- **No pictures of slides.** PDF and PowerPoint are exported from the finished HTML, never authored separately or stitched from screenshots.

No fallbacks: a missing input is a question to you, never a silent default.

THE NEWEST CAPABILITIES

What arrived most recently in the skill. Each is explained further on; the recipient needs no new tooling for any of them.

Edit SVG

- A drawn icon, diagram or chart is no longer a picture: its **parts** — paths, shapes, text, groups — move, resize, recolour, reorder, duplicate and reword in place.
- Path data is never rewritten; one reversible record per drawing.

One toolbar, three tabs

- **Text**, **Shape** and **SVG** share one floating panel; the tab of the editor in use comes to the front by itself.
- Pin, hide, anchor or detach it — remembered per deck.

PowerPoint export

- Any deck as an **editable .pptx** (1.25): real text boxes with their fonts and weights, shapes, cropped pictures, SVG pictures — never slide screenshots.
- Checked against the deck in **PowerPoint itself**, slide by slide.

The rebuild script

- Every deck ships with `<deck>.rebuild.mjs`: after a skill update it re-embeds the newer editing tools, keeps a backup, re-verifies and re-exports the PDF.
- `--check` only reports.

ALSO

27 bundled photos with a written catalogue (Athens at dusk, data centre, network, microchip, security, developer, team analytics — three takes each); assistant settings kept **per provider** and an *Ask about…* popup on any row of the Outline or Tree; a full set of **keyboard shortcuts** for text, shapes, SVG parts and panels; PDFs whose shadows render correctly in **macOS Preview** (1.19); **four themes** chosen per deck — NBG, BikS2013, AIHub, Instrument (1.20–1.23).

02

SECTION 02

Ask, build, edit

Asking for a deck, what happens in between, the quality gates, and everything the right-click editor can do.

NATIONAL BANK OF GREECE

INSTALLING THE SKILL

One public marketplace, two commands. No npm install, no build step: the skill is Markdown, assets and zero-dependency Node scripts.

IN CLAUDE CODE

```
# 1 · register the marketplace (once)
/plugin marketplace add BikS2013-coding-agents/nbg-design
# 2 · install the plugin from it
/plugin install nbg-design@nbg-design
# 3 · restart Claude Code, then check
/plugin list
# later · pick up a new release
/plugin marketplace update nbg-design
claude plugin update nbg-design@nbg-design
```

The marketplace is the public repository `github.com/BikS2013-coding-agents/nbg-design` (branch `working`); its name and the plugin's name are both `nbg-design`, hence `nbg-design@nbg-design`.

From then on Claude uses the skill by itself whenever a presentation in this look is asked for; say *nbg-design* to make sure.

WHAT LANDS ON THE MACHINE

The plugin is cached under `~/.claude/plugins/cache/nbg-design/nbg-design/<version>/`; the skill root is its `skills/nbg-design/` folder — the `<skill-root>` the pipeline commands use.

Requirements: Node 18 or newer for the scripts. Chrome, Chromium or Edge only for screenshots, the PDF and the PowerPoint export (its check also uses PowerPoint or LibreOffice, and poppler); without a browser the strict verifier is still the gate and the exports are made elsewhere.

Nothing to configure. The skill holds no credentials and reads no configuration file; assistant keys are entered by each viewer in their own browser.

A delivered deck outlives the install: after an update, `node <deck>.rebuild.mjs` re-embeds the newer editing tools — pointed at the new cache folder with `--scripts` or `NBG_DESIGN_SCRIPTS` when the recorded one is gone.

ASKING FOR A DECK

What the skill needs from you, what it decides by itself, and what it refuses to do.

It always asks for

- **Topic** — what the deck is about.
- **Audience** — who reads it; this sets depth and tone.
- **Slide count or depth** — 8, 16, 20+.
- Whether the bundled design-system files may be changed (they normally are not).

Approved defaults

- **Language**: English. `gr` and `bi` exist; anything else must come from you.
- **Output**: HTML. A PDF or a PowerPoint file is always exported from the finished HTML, never authored separately.
- **Logo**: shown, unless you ask to hide it.
- Cover first, dividers between sections, content slides in between.

It will not

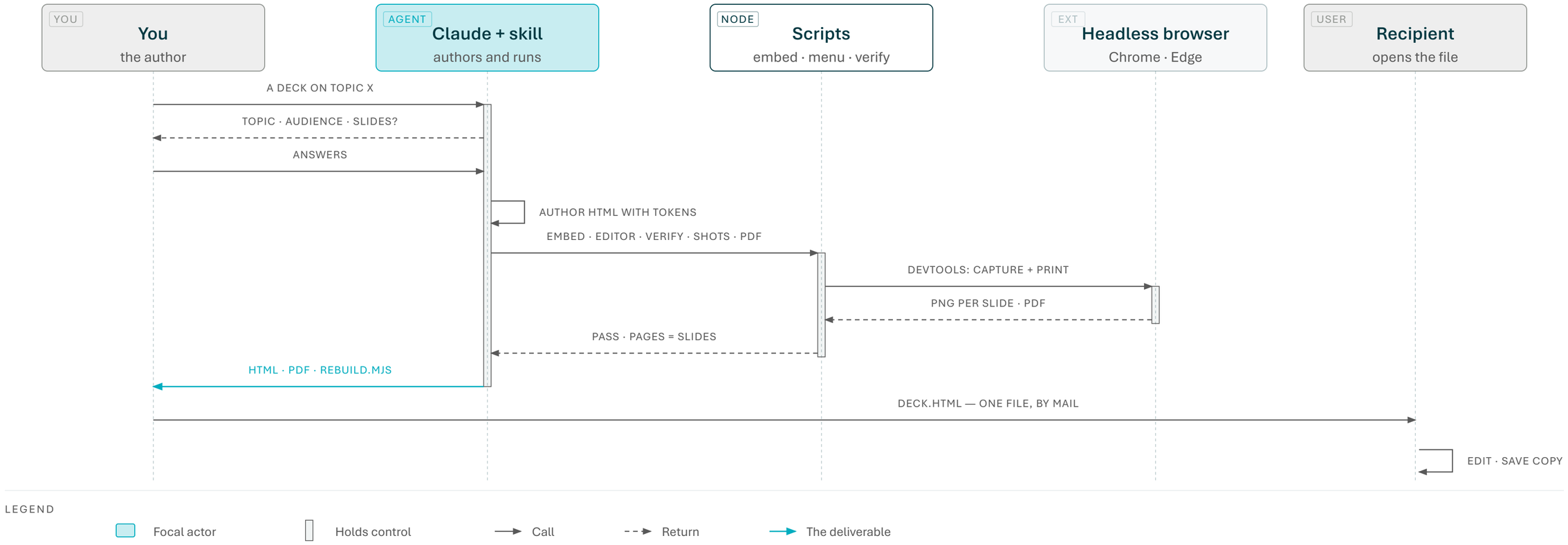
- Hand over a **PowerPoint made of slide pictures**: the .pptx is converted element by element.
- Guess a missing input or substitute a fallback value.
- Put an API key, token or endpoint into a deck or into the skill.
- Reference an image by a path: every image is embedded, or the deck fails the gate.

ANY CONTENT IS A SOURCE

A brief typed in chat, meeting notes, a Word or Markdown document, a spreadsheet, a report, an existing HTML deck, or a Google Slides draft (read through the Slides API and rebuilt 1:1, page count kept). The skill turns whatever it is given into a cover, dividers and content slides; it only asks for what it cannot infer.

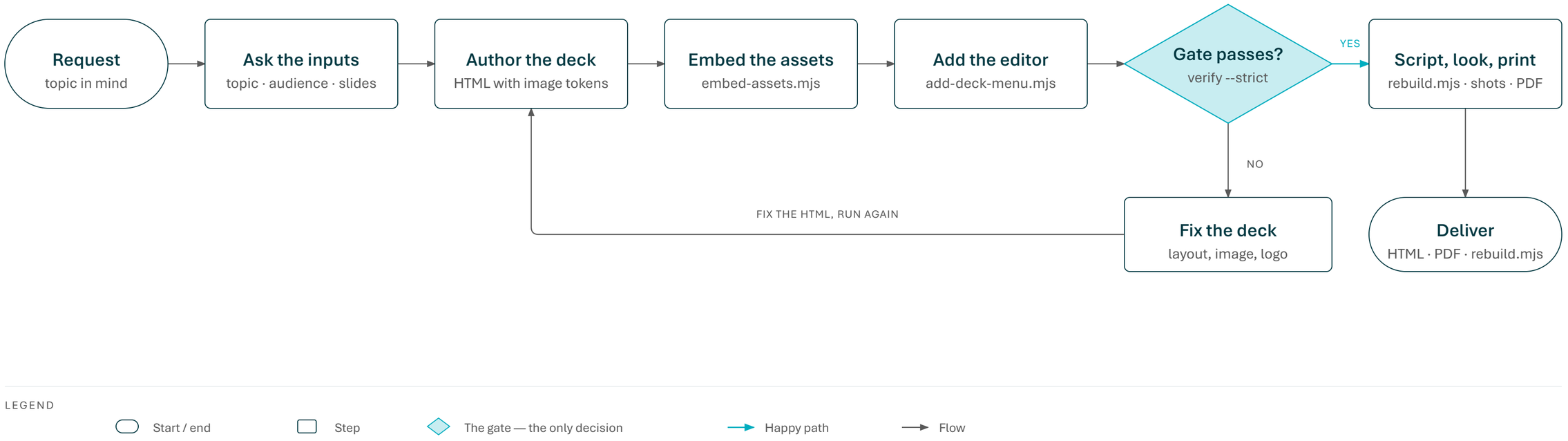
A DECK, END TO END

One request, three answers, one command chain, two files — and a recipient who can edit the result.



THE PIPELINE: FIVE STEPS AND ONE GATE

Claude authors; the scripts embed, add the editor and verify; a failing gate sends the deck back to the author, never forward.



THE PIPELINE, STEP BY STEP

What each step does and the command behind it. Steps 2 to 4 and the rebuild script are mandatory for every deck.

1

Author with tokens

Claude writes the deck on the 1920×1080 artboard, the design-system CSS and **placeholders** where images go: `{{LOGO_PRIMARY}}`, `{{PHOTO_STREET}}`.

2

Embed the assets

Every token becomes the verbatim base64 image. The script finds the assets next to itself, so it works from any folder and any machine.

```
embed-assets.mjs deck.html
```

3

Add the editor

One script block is inlined before `</body>`: the right-click menu, editing, toolbars, assistant, print. Re-running upgrades an older block.

```
add-deck-menu.mjs deck.html
```

4

Verify (strict)

Browser-free gate: no unresolved tokens, no external image paths, real logo, enough images, current menu. A failing deck is never delivered.

```
verify-deck.mjs deck.html --strict
```

5

Script, look, export

The rebuild script is written next to the deck; screenshots are read for overlaps and clipping; the PDF is printed, and on request the PowerPoint file converted and compared.

```
screenshot-deck · export-pdf · export-pptx
```

WHY TOKENS Hand-pasting a 600 KB base64 photo is the step that silently fails on a headless run. Tokens make embedding a script's job, and the gate catches any slip.

THE DESIGN SYSTEM THE SKILL CARRIES

Inspired by the NBG brand and distilled into one HTML reference: nine colours, one typeface and nine templates.

Deep teal
`#003841`
primary accent, dark panels, covers

Teal
`#007B85`
secondary accent, rules, icons

Bright cyan
`#00ADBF`
highlight accent

Electric cyan
`#00CFE7`
highlight, sparingly

Black
`#0A1416`
body text on light

Grey 1
`#BEC1BE`
rules, muted surfaces

Grey 2
`#939793`
secondary text

Cream
`#F5F8F6`
light page background

White
`#FFFFFF`
text on dark, cards on cream

Format & type

16:9, 1920×1080 artboard, scaled to fit any viewport. **Aptos** for everything, with a system fallback where it is absent — the stack is never swapped.

Nine templates

Covers (hero photo, image card, alternate), **dividers** (photo, deep teal, bright), **content** (image right, two columns, stat-led). Content pages stay calm and mostly monochrome; accent blocks belong to covers and openers.

Eight assets

Three logo lockups — **primary** for light backgrounds, **knockout** for dark, **small** for footers — and five photographs, each shipped with a ready-made data URI. The logo is always the image, never text or a box.

QUALITY GATES

What must be true before a deck leaves. The first gate runs anywhere; the others need a browser.

VERIFY-DECK --STRICT (BROWSER-FREE, MANDATORY)

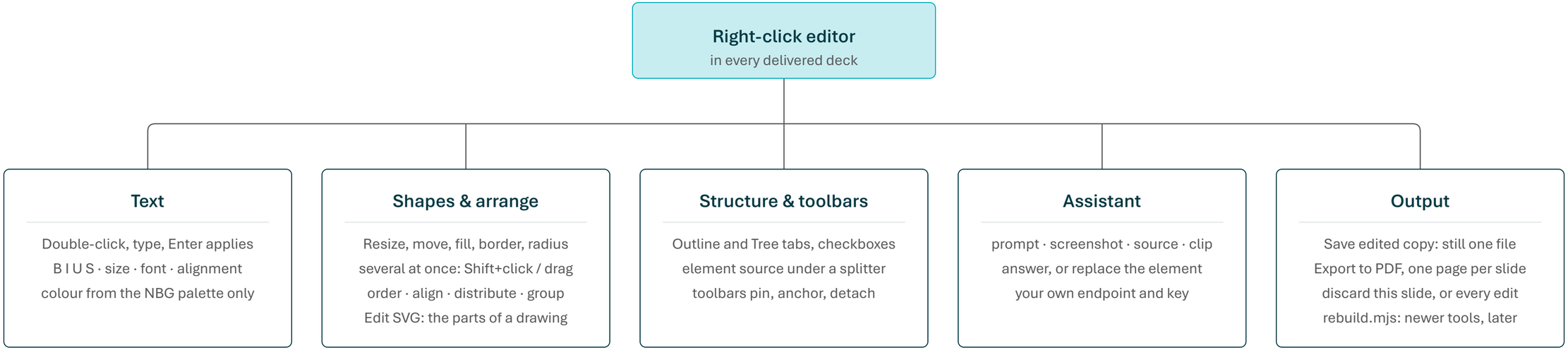
- **No unresolved tokens** and no image referenced by `file://`, absolute, relative or `http` path — every image is a `data:` URI.
- **Enough embedded images** (default 2) and a minimum size (200 KB): a photo-less deck is the classic tell of a skipped step.
- **A real logo**: bare `"NBG"` text nodes that could stand in for the lockup fail the gate.
- **The current editor block**: missing, PDF-only or older than the skill's — fail or warn; incomplete — always fail.

VISUAL AND PRINT CHECKS

- **Fit**: the full slide visible at 1366×768 and 1440×900 with no scrolling caused by the artboard.
- **No collisions**: title, body, card rows, callouts, footer and page number keep separate zones — at least 32 px between groups and 72 px above the footer. Crowded content is cut or split, never squeezed or hidden.
- **PDF = deck**: page count equals slide count and every slide box equals the page box, or the exporter exits with an error.
- **PowerPoint ≈ deck**: compare-pptx renders the .pptx in PowerPoint and scores every slide against the HTML.
- **Read the pictures**: screenshots, PDF pages and comparison images are inspected, not just produced.

WHAT A VIEWER CAN DO

Five families of actions, all in the file itself, all recorded and reversible. The slides that follow walk through them, and end with every keyboard shortcut.



Fonts and colours stay inside the NBG-inspired look: a viewer can fix a slide, not leave the system.

Images, new elements and new layouts remain the skill's job — regenerate.

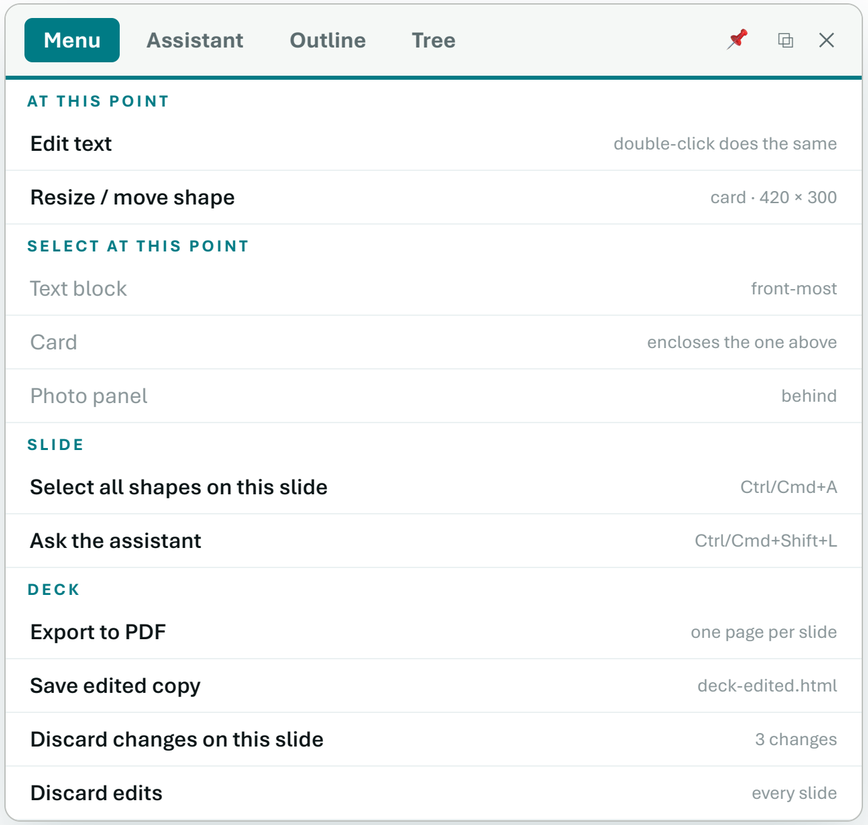
LEGEND

The editor itself

A family of actions, all recorded and reversible

EDITING 1 · THE RIGHT-CLICK MENU

Every delivered deck carries its own editor. Right-click anywhere on a slide; Escape or a click outside closes it.



It shows what applies to the spot you clicked. Text under the pointer offers *Edit text*; a card, image or panel offers *Resize / move shape*; a stack of nested shapes lists every level so you can pick the one you mean.

Four tabs. *Menu* holds the actions. *Assistant* is the AI panel. *Outline* lists the slide's shapes with checkboxes. *Tree* shows the slide's HTML, with the selected element's source underneath.

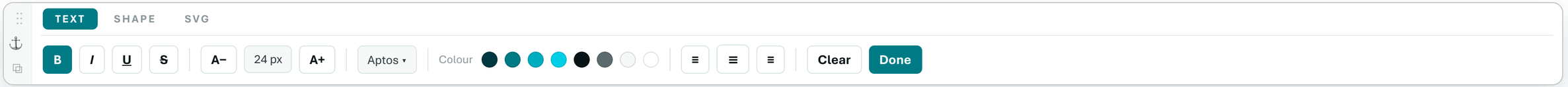
📌 **pins it** — it stays open after a choice, keeps its place and can be dragged by its header. ⧉ **detaches it** into a window of its own (always-on-top in Chrome and Edge).

Toolbars section. Ticks show which toolbar tabs are shown and which is in front; tick one to pin it and bring it forward, untick to hide, *Automatic toolbars* restores the default where they appear with the selection.

Lives outside the slides. The menu, frames and toolbars never appear in screenshots, in the PDF or in the print layout.

EDITING 2 · TEXT

Double-click any text. The whole block becomes editable in place, with a teal outline; the slide's CSS is never touched.



EDITING

- **Double-click** a text (or right-click → *Edit text*). Type, `Backspace`, `Shift+Enter` for a line break, `Ctrl/Cmd+Z` to undo.
- `Enter` or a click outside **applies**; `Esc` **cancels**. Deck shortcuts (arrows, space) are paused while you edit.
- **Paste and drop insert plain text.** Anything a browser wraps around typed text is unwrapped when the edit is applied, so no stray formatting leaks in.
- Accent spans inside a title (the teal word in a heading) stay intact.

THE FORMATTING ROW

- **Selection first.** Every control applies to the selected run when there is one, and to the whole block when there is none.
- **B / I / U / S** (`Ctrl/Cmd+B` `I` `U`), **A– / A+** or an exact px size (`Ctrl/Cmd+Shift+>` / `<`), **font** from the design-system stacks, **colour** from the skill's palette only, **alignment** for the block.
- **Clear** removes the selection's formatting, or returns the block to its authored style. **Done** applies.
- Fonts and colours are limited on purpose: a viewer can fix a slide but cannot leave the look.

EDITING 3 · SHAPES

Right-click a card, photo panel, image or text block → Resize / move shape. A teal frame with eight handles appears.



MOVING AND RESIZING

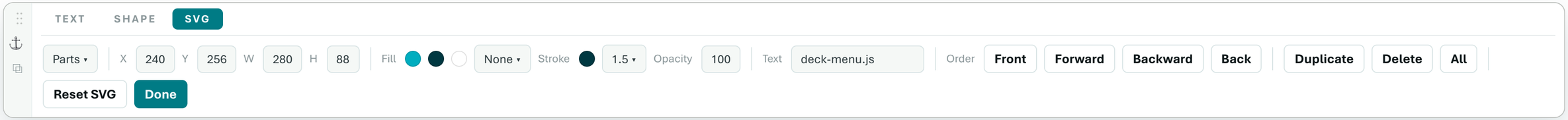
- **Drag a handle** to resize (Shift keeps the proportions), **drag inside** to move. Arrow keys nudge by 1 px, Shift by 10; Alt +arrows resize.
- Tab selects the enclosing shape, Shift+Tab steps back in; Esc, Enter or a click outside finishes.
- **Only inline geometry changes** — left, top, width, height — and pointer movement is divided by the slide's current scale, so coordinates stay exact artboard pixels at any window size.
- Elements in normal flow get only the right and bottom handles: they grow right and down. **Reset** restores size, position and style.

THE SHAPE ROW

- **X / Y / W / H** typed exactly; **fill** (the palette, transparent or default); **border** none to 6 px in a palette colour; **corner radius**, **opacity**, **shadow** none / soft / strong.
- Everything is inline style on the element, recorded as one reversible edit per element.
- **Neighbours do not re-flow.** Widen one card in a row, then use *Distribute* or move the next card to restore the spacing. The overlap rules still apply to the result.
- **Stack** lists the enclosing shapes, the selected one and its children, for the level a click cannot reach.
- **Edit SVG**: a drawn icon, diagram or chart has parts of its own — the next two slides. The SVG as a whole is still a shape: *Resize / move shape* sizes and places it.

EDITING 4 · EDIT SVG: THE PARTS OF A DRAWING

Right-click a drawn icon, diagram or chart → Edit SVG. The drawing gets a dashed outline and every part inside it becomes a thing you can pick.



ENTERING AND SELECTING

- **Three ways in**: right-click the drawing → *Edit SVG*; double-click a drawing that is already selected as a shape; click one of its parts in the **Tree** tab.
- **What a part is**: a path, rect, circle, ellipse, line, polygon, text, image, `use` reference or group. Nothing inside `defs`, clip paths, masks or symbols.
- **A click picks the innermost part** under the pointer — through the frame of a selected group too, so its members are one click away. **Shift** +click adds a part (or removes a selected one); **Ctrl/Cmd** +click selects what is inside the part there; **Ctrl/Cmd+A** or **All** every top-level part.
- **Tab** selects the enclosing group, **Shift+Tab** the first part inside; the **Parts** list offers every part in drawing order.

MOVING, RESIZING, STYLING

- The selection gets a frame with eight handles — several parts: one frame spanning them, a dashed mark per part. **Drag inside** to move, **a handle** to resize (**Shift** keeps proportions); arrows nudge by one SVG unit, **Shift** ten, **Alt** +arrows resize — all selected parts together, keeping their places.
- **The SVG row**: X / Y / W / H in the drawing's own units; **fill** and **stroke** from the palette, *None* or *Default*; **stroke width**; **opacity**; the **text** of a label (one run; Enter applies).
- **Order** in the drawing (**Ctrl/Cmd+]** / **[**), **Duplicate** (**Ctrl/Cmd+D**; copies sit on the originals), **Delete** (**Delete**), **Reset SVG** (every part as designed), **Done**.
- **Esc** deselects the part, then ends the session; **Enter**, *Done*, *Finish SVG editing* in the menu or a click outside the drawing end it too.

EDITING 5 · WHAT AN SVG EDIT WRITES

Geometry goes into attributes, colour into presentation attributes, and the whole drawing becomes one reversible record. The path data itself is never touched.

Geometry, never path data

- A move or resize is written in the part's **parent coordinate system**: the screen box is mapped back through the parent's transform.
- A rect or image without a transform gets plain `x / y / width / height`; a pure move of text, a `use`, a circle, an ellipse or a line changes `x / y`, `cx / cy` or `x1…y2`.
- Everything else — a path, a scaled text, a group — gets a `matrix()` **prepended** to its own transform. The `d` attribute stays as drawn.

Colours, order, text

- Fill, stroke, stroke width and opacity become **presentation attributes**, or the inline style property when the part already sets one — so the change wins over the stylesheet without touching it.
- Order is a real move in the drawing order inside the parent; Duplicate clones without ids and places the copy after the original.
- The text field replaces the one run of a `<text>`; labels with several runs are edited as source in the Tree tab.

One record for the whole drawing

- Every operation records the `<svg>` element's inner HTML as a single **html** edit — the same kind as a text edit — so it persists in the browser, lands in the saved copy, is discarded per slide and prints.
- **Reset SVG** drops that record. Sessions are exclusive: editing text, selecting a shape or preparing a print ends the SVG session first.
- SVG text is never `contenteditable`; the SVG as a whole is still a shape for *Resize / move shape*.

KNOWN LIMITS

The frame is the axis-aligned screen box: a part inside a **rotated** group is moved correctly in that group's space, but its frame does not rotate. Parts sized in percentages or with a CSS transform take the matrix path. The five diagrams in this deck are editable this way.

EDITING 6 · SEVERAL SHAPES AT ONCE

Text blocks, cards, panels and images alike. The frame spans the selection and each member gets a dashed mark.

Selecting

- `Shift`+click adds the smallest shape under the pointer, or removes it when it is already selected.
- `Shift`+drag on the slide draws a box: every top-level shape fully inside is added.
- `Ctrl/Cmd+A` selects every top-level shape of the slide; `Ctrl/Cmd`+click selects everything inside the shape under the pointer; a shape with nothing inside is picked alone, even a group member, and on the same spot again cycles outward.
- The menu offers *Add to / Remove from selection / Select all shapes*; the **Outline** tab does it with checkboxes.

Arranging

- **Order**: bring to front / forward, send backward / to back (`Ctrl/Cmd+]` and `[`, `Shift` for front / back).
- **Align** left, centre, right, top, middle, bottom — relative to the selection's own box or to the slide.
- **Distribute** horizontally or vertically with equal gaps; the first and last stay put.
- Dragging moves all, handles scale all proportionally, every style control applies to all.

Grouping

- **Group** (`Ctrl/Cmd+G`) / **Ungroup** (`Ctrl/Cmd+Shift+G`).
- A group is logical: members carry a `data-nbg-group` mark and always select, move, resize, align and order together.
- Groups persist in the browser, land in the saved copy and are removed by *Discard edits*.
- The page structure is never rebuilt: ordering is an inline z-index, grouping an attribute, so every earlier edit stays valid.

NESTED SHAPES A click lands on one level only. Use **Select at this point** in the menu (front-most first, containers, shapes behind), the toolbar's **Stack** list, or `Tab` / `Shift+Tab` to reach the others.

EDITING 7 · STRUCTURE, WINDOWS AND THE PICKER

The Outline and Tree tabs, panels that leave the browser window, and the compact menu that takes over while the big one is away.

Outline & Tree

- **Outline**: the slide's shapes nested by containment — cards, text blocks, images — with kind icons, sizes, group badges, a text filter, checkboxes, All / None. Click a name to select it, Shift+click adds.
- **Tree**: the slide's HTML elements, synced both ways with the selection; double-click edits a text; hovering outlines the element on the slide.
- Under the tree, the selected element's **editable source** behind a draggable splitter. *Apply* records it like any other edit; scripts are stripped. For people who know what they are doing.

Detachable panels

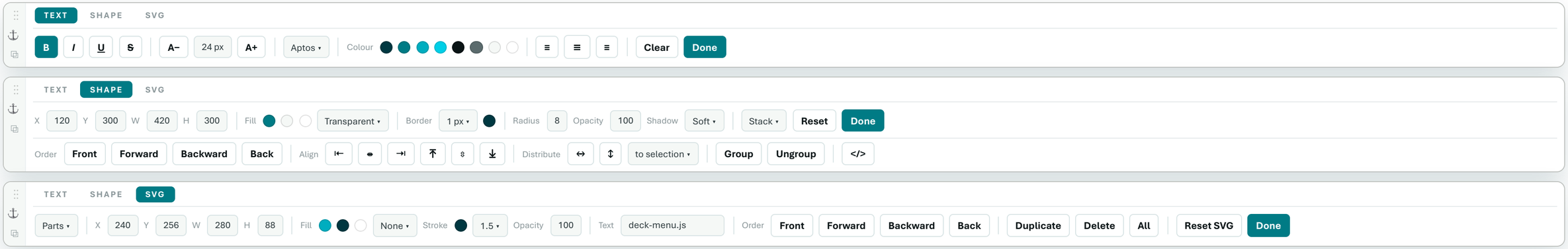
- ⧉ on the menu (structure and assistant included) or on the toolbar moves it into a **window of its own**: a Document Picture-in-Picture window in Chrome and Edge — always on top, no browser chrome, anywhere on the desktop, even off the browser — or a pop-up elsewhere.
- The panel keeps working on the deck from there; closing the window, ✕ or Esc brings it back.
- A detached panel is pinned: it idles rather than disappears when nothing is selected. Screenshots asked from a detached assistant use that window's permission.

The picker

- While the menu is detached, or open on a panel tab, a **right-click on the slide opens a compact picker** at the pointer instead of the full menu.
- It holds the same *Select at this point* hierarchy — front-most first, containers and shapes behind — plus *Edit text*.
- A click selects, `Shift`+click adds, `Esc` closes it. The big menu keeps its place and its tab.
- `Ctrl/Cmd+Shift+O` / `H` open Outline / Tree directly; `Ctrl/Cmd+Shift+L` the assistant.

EDITING 8 · THE TOOLBAR: ONE PANEL, THREE TABS

Text, Shape and SVG share one floating panel with one grip, one anchor and one detach button. One tab's row shows at a time.



The editor in use takes the front. Text being edited, or a selection made only of text blocks → **Text**; any other selection → **Shape**; a drawing in SVG editing → **SVG**. The panel appears with the selection and follows it.

A tab you click stays in front until another editor takes over. A tab whose editor has nothing to work on is **dimmed but opens**: its row idles with the controls greyed.

Three modes per tab — auto (with the selection), on (pinned), off (hidden until shown again) — set from a row's ✕ or the menu's *Toolbars* section, and remembered per deck in that browser.

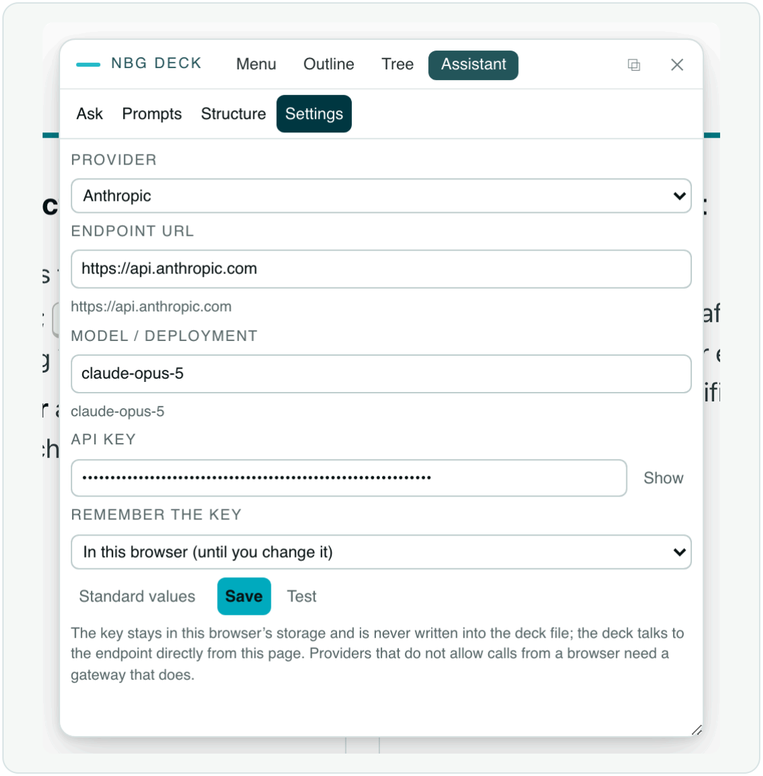
Anchored: drag the panel by its grip and it opens at that spot from then on, showing ⚓; click ⚓ or double-click the grip and it follows the selection again.

Detached: ⧉ moves the whole panel, tabs included, into a window of its own (always on top in Chrome and Edge). It keeps working on the deck; closing the window brings it back.

Every tab adapts to the selection: the Shape row shows the selection's geometry and style, the SVG row the part's units, and the Text row formats selected text blocks without entering edit mode.

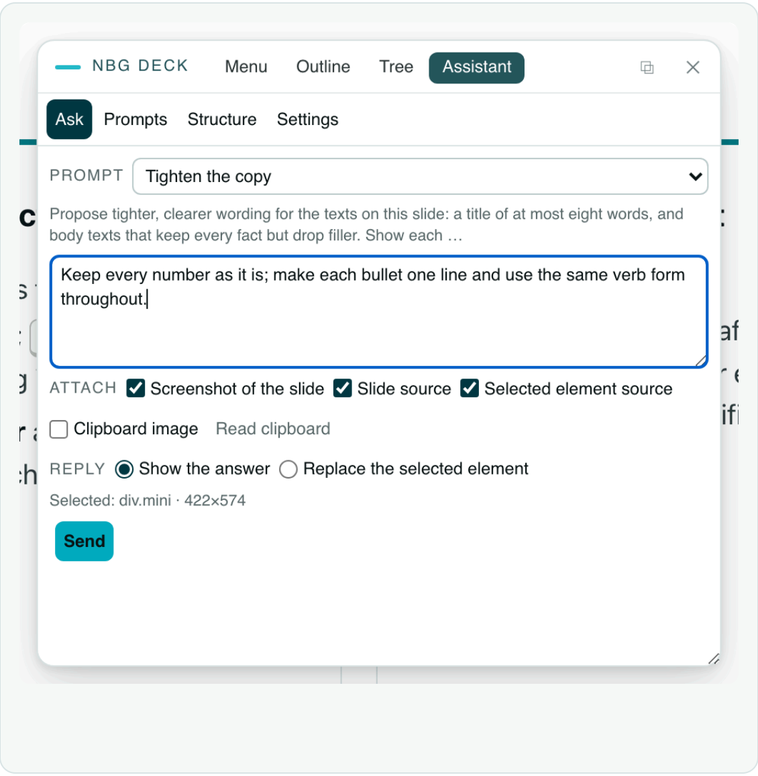
EDITING 9 · THE ASSISTANT, AS IT LOOKS

Three real screenshots from this deck: the Settings view, the Ask view with a card selected, and the Ask-about popup opened from a row of the Outline.



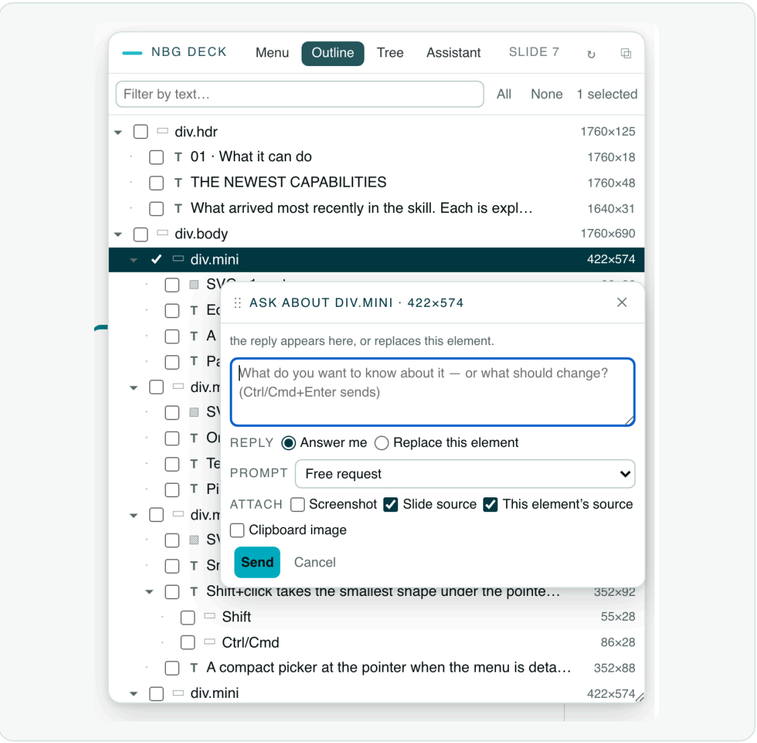
Settings

Provider, endpoint, model or deployment, API key and where it is remembered. Settings are kept per provider; *Standard values* fills a fixed endpoint, *Test* sends a one-line request. The key never enters the deck file.



Ask

A prompt from the drop-down with its text shown beneath, a request box, the four attachments as checkboxes, the reply mode, and what is selected. *Send* shows the answer here, or replaces the selected element with Undo.



Ask about… on a row

Right-click a row of the Outline or Tree: a compact popup for that element, with its text, a request box, *Answer me / Replace this element*, the prompt and the attachments. It stays put until you close it and never opens the panel by itself.

EDITING 10 · ASK THE ASSISTANT

An AI request from inside the deck, with the slide as context. The viewer brings their own endpoint and key; nothing is baked in.

Ask

- A **prompt** from the drop-down: *Free request*, *Review the slide*, *Proofread the text*, *Tighten the copy*, *Speaker notes*, *Restyle the selection*, *Rewrite the selection's text*, plus your own.
- A request box for extra instructions.

Attach

- **Screenshot of the slide** (tab capture; toolbars hidden, cropped to the slide).
- **Slide source** — its HTML plus the CSS rules it uses; embedded images travel as small placeholders, so photos cost no tokens.
- **Selected element source.**
- **Clipboard image** — paste, drop or *Read clipboard*.

Reply

- **Show the answer** in the panel, with *Copy* and *Apply to selection*.
- **Replace the selected element**: the reply is the element's replacement HTML — same tag, scripts stripped, recorded as a normal edit — with **Undo**.
- Right-click a row in Outline or Tree for an *Ask about…* popup on that element.

Settings

- **Providers**: Anthropic, Azure-hosted Anthropic (Microsoft Foundry), OpenAI-compatible, Azure OpenAI, DeepSeek.
- Endpoint, model or deployment, API key (kept in this browser, or for the tab only), *Test*.
- Settings are kept per provider; nothing is defaulted — a missing value blocks the request with a message.
- The page calls the endpoint directly; a provider that blocks browser calls needs a gateway.

PRIVACY Each attachment is a checkbox, so the viewer decides per request what leaves the machine. The key never enters the deck file or a saved copy.

EDITING 11 · KEEPING, SHARING AND PRINTING YOUR CHANGES

The deck on disk never changes by itself. Edits live in the browser until you save a copy.

Edits persist in the browser, keyed by the file: reload the deck and they are still there. They exist only in that browser — the original file is untouched.

Save edited copy downloads `<name>-edited.html` : your edits applied to a pristine snapshot of the deck taken at load. The copy is still one self-contained file and carries the editor itself.

Discard changes on this slide restores the slide under the pointer (the hint counts its changes); **Discard edits** restores every slide.

Refresh the tools later: `node deck.rebuild.mjs` replaces only the editor block with the one the skill ships now, keeps a backup, re-runs the strict gate and re-exports the PDF. `--check` only reports CURRENT or REBUILD NEEDED.

Export to PDF applies the same print layout as the command-line exporter and opens the print dialog: choose *Save as PDF* in Chrome or Edge. Page size, zero margins and backgrounds are preset — one page per slide, the same layout as the CLI export. Blurred shadows are printed as images on this path (block v13), so the PDF also renders correctly in macOS Preview.

WHAT THE EDITOR CAN AND CANNOT DO

IN THE DECK, BY ANYONE	BY REGENERATING, WITH THE SKILL
Text, formatting, size, font, palette colour, alignment	New images or photos
Geometry and style of shapes: fill, border, radius, opacity, shadow	New elements, new slides, new layouts
Order, alignment, distribution, groups of several shapes	Structural changes to the page
AI-assisted rewrites and restyles of an element	Anything outside the skill's fonts and palette

A PDF of an edited deck is printed from the saved copy — the CLI prints what is on disk.

KEYBOARD SHORTCUTS

Every shortcut the editor understands, by mode. Ctrl on Windows and Linux, Cmd on a Mac; deck navigation keys are paused while you edit.

EDITING TEXT

Enter	applies the edit; a click outside does the same
Esc	cancels the edit and its formatting
Shift+Enter	line break
Ctrl/Cmd+Z	undo while editing
Ctrl/Cmd+B **I** **U**	bold, italic, underline
Ctrl/Cmd+Shift+> **<**	larger / smaller text
Double-click	a text: start editing it

SHAPES (ONE SELECTED)

↑↓←→	nudge by 1 px; **Shift** 10 px
Alt+arrows	resize
Tab **Shift+Tab**	enclosing shape / child shape
Shift+click	add the smallest shape there, or remove a selected one
Shift+drag	selection box on the slide
Ctrl/Cmd+click	everything inside the shape; a leaf alone, again cycles outward
Ctrl/Cmd+A	every top-level shape of the slide
Ctrl/Cmd+] **[**	forward / backward; **Shift** front / back
Ctrl/Cmd+G **Shift+G**	group / ungroup
Esc **Enter**	finish

SVG PARTS

Click	innermost part; drag moves it
Shift+click	add a part, or remove a selected one
Ctrl/Cmd+click	what is inside the part there
Ctrl/Cmd+A	every top-level part
Tab **Shift+Tab**	enclosing group / first part inside
↑↓←→	nudge one unit; **Shift** ten; **Alt** resizes
Ctrl/Cmd+] **[**	drawing order; **Shift** front / back
Ctrl/Cmd+D	duplicate
Delete **Backspace**	delete the part
Esc	deselect, then end; **Enter** ends

MENU AND PANELS

Right-click	the menu, or the picker while the menu is away
Esc	closes the menu, the picker, a popup
Ctrl/Cmd+Shift+O	Outline tab
Ctrl/Cmd+Shift+H	Tree tab with the element's source
Ctrl/Cmd+Shift+L	the assistant
Ctrl/Cmd+V	paste an image into the assistant's request
Double-click	a shape selects it; a selected drawing opens its parts; a grip releases an anchored panel

03

SECTION 03

Make it your own

How it is built: the folder, the scripts and libraries, the editor's internals, the PDF and PowerPoint paths, the configuration hook and the rules to keep.

NATIONAL BANK OF GREECE

HOW IT IS BUILT, IN FIVE LAYERS

Each layer rests on the one below; extend the one you need. The editor is the layer that turns a deck into something recipients can change themselves.

BUILDS ON

L5	**Deliverables** One self-contained HTML deck, its PDF, its rebuild script	images embedded · one PDF page per slide · tools refreshable later
L4	**In-deck editor** Right-click menu, toolbars, structure, assistant	print-layout.js + deck-menu.js, inlined into the deck
L3	**Build pipeline** Embed · add editor · verify · screenshot · export	7 zero-dependency Node scripts + 5 shared libraries
L2	**Design system** Palette, Aptos, 1920×1080 artboard, 9 templates	3 logo lockups · 5 photos · ready-made data URIs
L1	**Contract** SKILL.md: inputs, defaults, guardrails, quality gate	one Markdown file is the whole behaviour

LEGEND

Focal layer — what makes a delivered deck editable by anyone

Layer — rests on the one below it

ANATOMY OF THE SKILL FOLDER

31 MB, 30 of them photography; one Markdown file that is the whole behaviour, nine scripts, seven libraries, and four themes with their assets.

```
nbg-design/  plugin nbg-design v1.25.0 · marketplace BikS2013-coding-agents/nbg-design · MIT
├─ SKILL.md  104 KB · 621 lines  the single source of behaviour, defaults, assets, guardrails
├─ NBG-Design/
│   ├─ NBG Design System.html  39 KB  visual reference
│   ├─ slide-templates.jsx  32 KB  nine 1920×1080 templates
│   ├─ tweaks-panel.jsx  24 KB  tweak / edit helper reference
│   ├─ assets/  14 MB  3 logos + 27 photos, each with a .datauri.txt
│   └─ screenshots/  8 reference renders
├─ BikS2013-Design/ · AIHub-Design/ · Instrument-Design/  the other three themes
├─ scripts/  660 KB  nine scripts, seven libraries
│   ├─ embed-assets.mjs · add-deck-menu.mjs · verify-deck.mjs
│   ├─ write-rebuild-script.mjs · screenshot-deck.mjs · export-pdf.mjs
│   ├─ fix-pdf-soft-masks.mjs · export-pptx.mjs · compare-pptx.mjs
│   └─ lib/  find-browser.mjs · cdp.mjs · print-layout.js
│         pdf-soft-masks.mjs · deck-menu.js  339 KB
│         pptx-extract.js · pptx-writer.mjs  93 KB
└─ references/  design decisions, functions, config guide, issue register
```

- **SKILL.md is the contract.** Inputs, defaults, palette, templates, embedding rules, the editor's behaviour, the PDF and PowerPoint rules and the quality checklist — all in one file, no separate configuration.
- **Everything resolves relative to the skill root**, never to the working directory: the scripts find the assets next to themselves, so the skill works from any folder, on any machine, through a symlink.
- **Installed as a project skill** via a symlink in `.claude/skills/` pointing at the plugin checkout, so the working tree is the live skill.
- **References are history, not behaviour**: every design decision from v1.3 to v1.25 is recorded with its rationale, mechanism and verification in project-design.md.

THE SCRIPTS AND LIBRARIES

Zero-dependency Node (18+). No npm install, no build step; nine single-file scripts that import from lib/.

SCRIPT	WHAT IT DOES	NEEDS	EXIT CODES
`embed-assets.mjs`	Replaces every `{{TOKEN}}` with the verbatim data URI; tokens map to files by name; fails loudly on a missing asset.	none	0 / 1
`add-deck-menu.mjs`	Inlines one `<script id="nbg-deck-menu-script" data-nbg-deck-menu="17">` before the last `</body>`: print-layout.js then deck-menu.js. Idempotent; upgrades older blocks; `--remove` strips.	none	0 / 1
`verify-deck.mjs`	The gate: tokens, image paths, image count, size, bare-logo text, editor block version. `--strict` promotes the warnings to failures.	none	0 pass / 1 fail
`write-rebuild-script.mjs`	Writes `<deck>.rebuild.mjs` next to the deck: a self-contained script that later re-embeds the skill's current editor block (backup, verify, re-export the PDF); `--check` only reports.	none	0 / 1 / 2
`screenshot-deck.mjs`	Drives a headless browser over the DevTools protocol; isolates the Nth `.slide` in place, so `.active`, `.hidden`, inline-display and stacked decks all work; PNG per slide per viewport.	browser	0 / 1 / 3 no browser
`export-pdf.mjs`	Lifts the host layer, prints with `Page.printToPDF` at the slide box, verifies pages = slides and slide box = page box; then re-anchors the soft masks so shadows render in macOS Preview.	browser	0 / 1 / 3 no browser
`fix-pdf-soft-masks.mjs`	Re-anchors the soft masks of a PDF exported earlier (or saved from the print dialog): an incremental update, the original bytes intact; `--check` only reports.	none	0 / 1 / 2
`export-pptx.mjs`	Measures every slide in the browser and writes an editable .pptx: text boxes with real runs, shapes, cropped pictures, SVG pictures, the theme's fonts and colours.	browser	0 / 1 / 3 no browser
`compare-pptx.mjs`	Renders the .pptx with PowerPoint (macOS) or LibreOffice and writes deck | PowerPoint | difference images with a score per slide.	browser	0 / 1 / 3 missing tool

lib/find-browser.mjs

Locates Chrome, Chromium or Edge; `--browser`, `NBG_BROWSER`, `CHROME_BIN` override.

lib/cdp.mjs

A minimal DevTools-protocol client over the debugging pipe: launch, navigate, evaluate, print, count pages.

lib/print-layout.js

The print shim shared by the exporter and the in-deck menu; a registry undoes every change after printing. On the menu path it prints blurred shadows as images (v13).

lib/pdf-soft-masks.mjs

Zero-dependency PDF rewriter: re-anchors Chrome's luminosity soft masks so macOS Preview renders shadows; incremental update.

lib/deck-menu.js

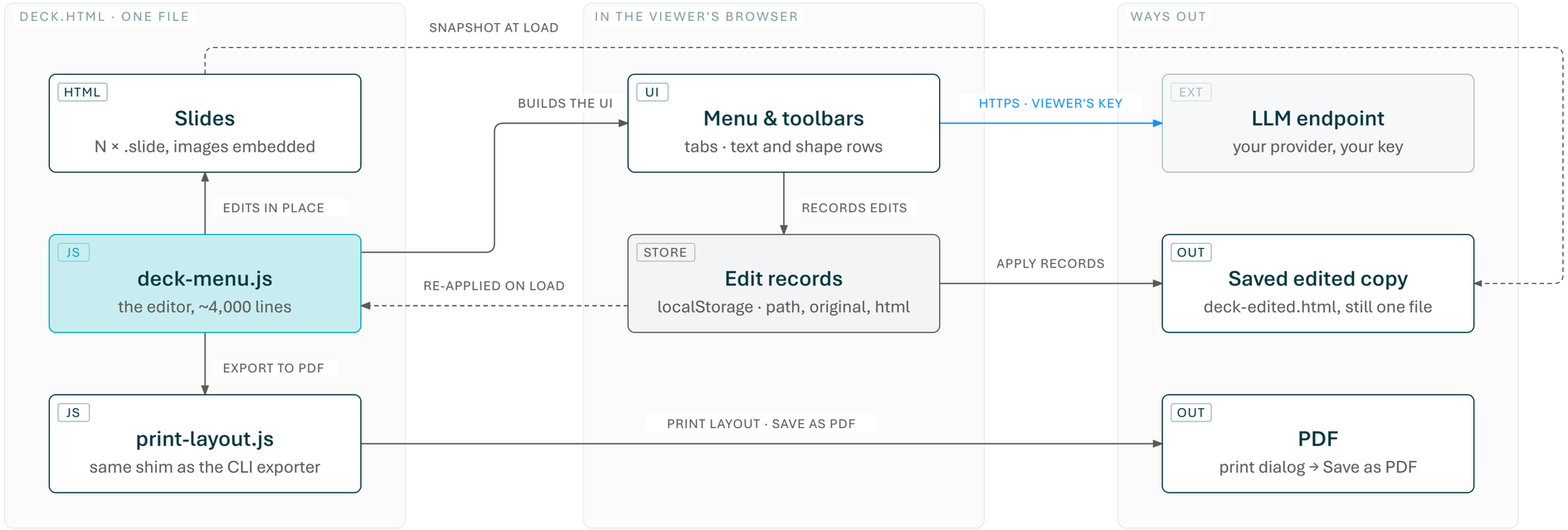
The editor: ~4,000 lines of plain browser JavaScript — editing, selection, SVG parts, toolbars, structure, assistant, persistence, print.

lib/pptx-*

pptx-extract.js measures every slide in the browser; **pptx-writer.mjs** writes the Office Open XML and its zip.

INSIDE A DELIVERED DECK

One HTML file carries the slides and the editor; the browser keeps the edits; three ways lead out.



LEGEND

Focal component · Browser storage · Outside the deck · Calls / writes · Passive: taken or re-applied on load · HTTPS

INSIDE THE EDITOR

How a standalone file:// deck can be edited, remembered and handed back without ever writing to itself.

Edit records

- Every change is a record `{{ path, original, … }}` where **path** is the child-index path from the root — cheap, exact, and the reason the DOM is never restructured.
- Three kinds: **html** (text and selection-level formatting), **style** (block formatting, geometry, fill, border, z-index), **group** (the `data-nbg-group` attribute). An SVG session records the whole `<svg>` as one html record.
- Stored in `localStorage` under `nbg-deck-edits:<path>#<title>`; re-applied on load; a record whose original no longer matches is dropped.

Pristine snapshot

- When the inlined script runs — after the deck's own scripts, before any runtime element and before stored edits — it takes an `outerHTML` snapshot.
- **Save edited copy** parses that snapshot with `DOMParser`, applies the records, and downloads the result: it loads exactly like the original and carries the menu.
- The print path uses the same print-layout shim as the CLI, then restores every class, style and attribute it touched.

Geometry, order, groups

- A "shape" is a positioned element, one with a background, border or shadow, or an image; otherwise the text block.
- Pointer deltas are divided by the slide's scale (`rect.width / offsetWidth`), so coordinates are artboard pixels.
- Ordering ranks siblings by effective layer and writes an inline `z-index`; a negative index makes the parent a stacking context with `isolation: isolate`.
- Rich `contenteditable`, not `plaintext-only`, because the latter forces `pre-wrap` and re-flows titles.

DESIGN RULE Inline style and attributes only, never DOM re-ordering — so every earlier record's path stays valid and every change stays reversible.

THE PDF PATH

The PDF is the deck, printed by the same engine that renders it. Nothing is re-laid out for paper.

WHAT THE EXPORTER DOES

- Opens the deck in headless Chrome, Chromium or Edge over the DevTools pipe — nothing to install.
- **Lifts the host layer only**: navigation toggles (`.active` , `.hidden` , inline display), the viewport-fit wrappers and scaling, buttons and counters; jumps animations to their end state; forces `print-color-adjust: exact` .
- Makes every top-level `.slide` visible in flow and prints with `Page.printToPDF` : page size = the measured slide box (1920×1080 px = 20 × 11.25 in), zero margins, backgrounds on, scale 1.
- **Verifies**: page count equals slide count and every slide box equals the page box; otherwise exit 1 (the PDF is still written for diagnosis). Exit 3 means no browser on the host.
- **Re-anchors the soft masks** (since 1.19): Chrome writes every blurred shadow as a luminosity mask that macOS Preview mis-positions into gray blocks. The exporter rewrites them to page space as an incremental update; every other viewer draws the same picture. `fix-pdf-soft-masks.mjs` does it for older PDFs.

RULES THAT ARE NOT NEGOTIABLE

- **Aesthetics are frozen.** No print stylesheets, no resized slides, no font swaps "to make it print". A mismatch is a bug in the deck, fixed in the HTML.
- **No fakes.** No `window.print()` at A4, no browser Save-as-PDF with headers and margins, no office round-trips, no stitched screenshots.
- Every top-level slide carries the `slide` class — the exporter and the screenshot helper key on it.
- **Fonts come from the exporting host**: with Aptos absent, PDF and screenshots share the same fallback. Export where Aptos is installed if it must appear.
- Never print a deck that fails `verify-deck --strict` . The in-deck *Export to PDF* inlines the very same shim; since the print dialog's PDF cannot be post-processed, that path prints blurred shadows as images instead (block v13).

THE POWERPOINT PATH

An editable .pptx, converted element by element from the deck the browser lays out. Never a screenshot per slide.

WHAT EXPORT-PPTX DOES

- Opens the deck in headless Chrome with the PDF exporter's print layout, then **measures** every element where the browser put it — positions are Chrome's, never re-derived.
- **Text → text boxes** with real runs: typeface and the weight actually drawn (*Aptos Light*, *Avenir Next Ultra Light*), size, colour, letter-spacing, capitals, raised runs, links, line breaks.
- **Boxes → shapes**: fills and gradients, borders, per-corner radii, shadows, transparency. **Photos → pictures** with their full source and a crop. **Inline SVG → SVG pictures**, convertible to shapes in PowerPoint.
- The theme carries the deck's fonts and palette, so new text and PowerPoint's colour picker match the deck.

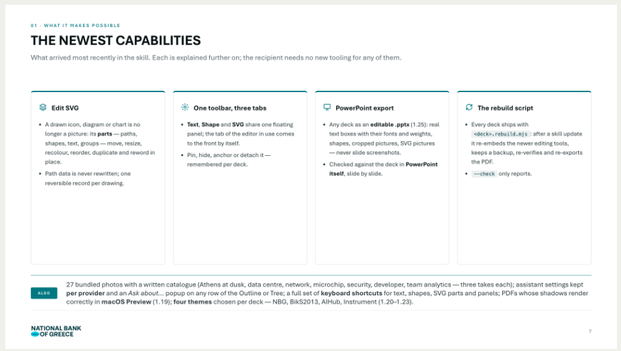
WHY IT LANDS WHERE THE HTML DOES

- **Calibrated against PowerPoint**: its first baseline sits at 0.75 × the line pitch, so each box is placed to put it on Chrome's; the pitch is written as a percentage, because exact spacing is rounded to whole points.
- **Chrome's line breaks are kept** where a greedy renderer cannot find them: balanced titles, paragraphs with code or key chips.
- **compare-pptx** renders the file in PowerPoint itself and writes deck | PowerPoint | difference images with a score per slide — read, not just produced.
- **Fonts are named, not embedded**: the presenting machine needs the deck's faces. Aptos ships with Office.

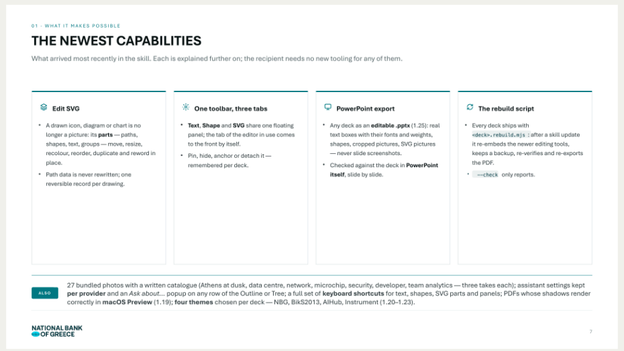
THE POWERPOINT FILE, CHECKED

Two slides of this very deck: the HTML, its .pptx as rendered, and the pixels that differ. compare-pptx writes one such image per slide.

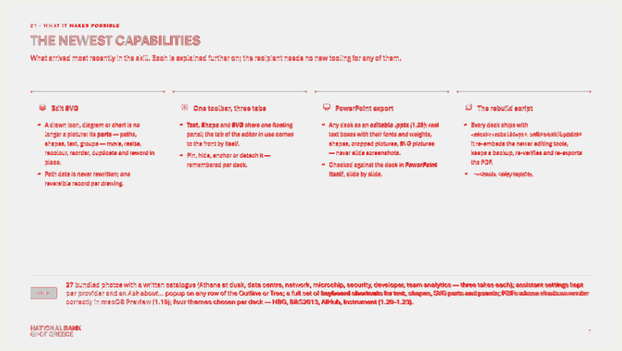
Slide 7 — HTML deck



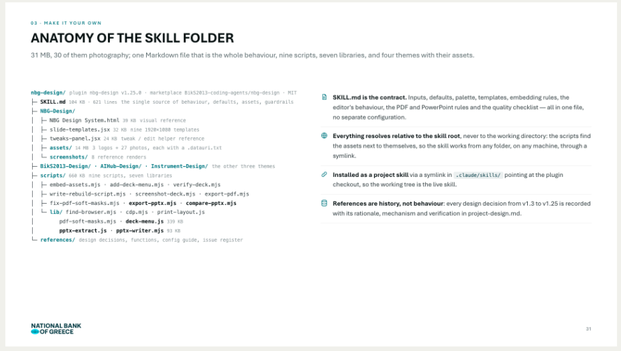
.pptx rendered by LibreOffice



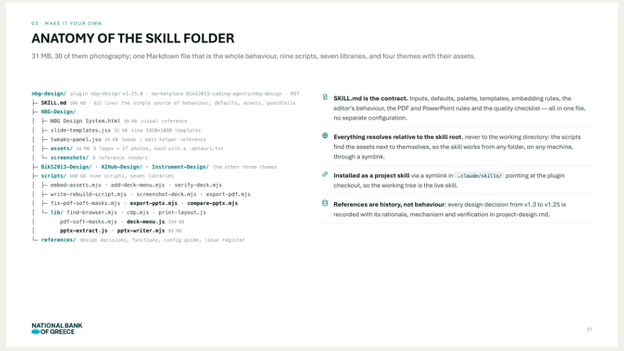
Difference — 0.74 % solid



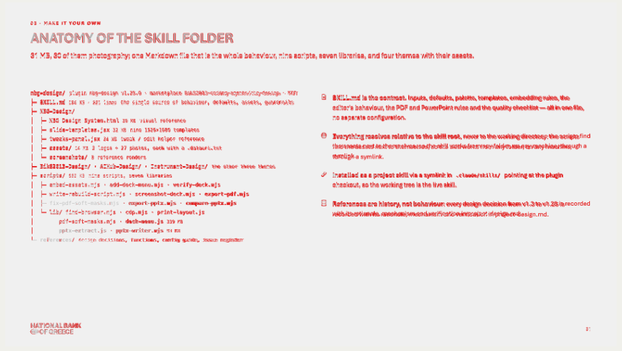
Slide 31 — HTML deck



.pptx rendered by LibreOffice



Difference — 0.69 % solid



HOW TO READ IT

Left the deck, middle the .pptx (here rendered by LibreOffice), right the differences in red. Below about 0.5 % solid difference is anti-aliasing and sub-pixel font metrics; text that re-wraps or a shape out of place would show as a red block.

THE CONFIGURATION HOOK: ONE EDITOR FOR ANY HTML PAGE

Since v1.15 the editor is configured, not forked. A deck keeps its defaults; any other document declares what its editable regions are.

WINDOW.NBGDECKMENUCONFIG

```
window.nbgDeckMenuConfig = {
  mode: 'deck' | 'page',  print: one page per slide, or the browser's pagination
  root: '.slide',  CSS selector of the editable regions (top-level matches)
  unit: 'Slide',  the word the UI uses: Slide, Page, Section …
  title: '…',  the label in the menu header
  aiSystem: '…'  replaces the assistant's system prompt
};
```

- Written as the block's first statement by `addMenu(html, config)` / `buildMenuBlock(config)`; `readMenuConfig(html)` reads it back. The CLI has no flags: a deck always gets the defaults.
- **Page mode**: the print path adds only background and animation rules; wording says *page* / *Section N*; the assistant gets page-oriented prompts (Review the page, Summarise the page …).

REUSING THE SKILL IN GENERATORS

- A deck can be a **data-driven Python generator**: a helper section (icons, header, footer, pills, numbered circles) and one `add()` per slide, sharing one `deck.css` and `deck.js` with sibling decks.
- Slides can be **sliced out of another generator** by comment markers and re-executed, so a slide reused across decks is one line.
- A `rebuild.sh` per deck detects skill, generator or asset changes, builds the next version through the five steps and refreshes a fixed-name `-latest` copy — the stable link.
- This deck was built that way: a 550-line generator, the shared CSS plus 150 lines of additions, and the pipeline.

RULES THAT KEEP IT SAFE

The constraints written into SKILL.md that every generated deck, and every future change to the skill, must respect.

Delivery

- Run order: author → embed → add menu → verify strict → rebuild script → screenshots → PDF / PowerPoint. Never deliver a deck that fails the gate.
- Every image a `data:` URI; the logo always the bundled lockup.
- Fit at 1366×768 and 1440×900; no element collisions; 32 px between groups, 72 px above the footer.
- PDF and PowerPoint only from the verified HTML, only through the exporters; the .pptx always compared.

Secrets & fallbacks

- The skill holds **no credentials**. Assistant keys are the viewer's and live only in their browser — never in the skill, a deck or a saved copy.
- **No fallback values**: a missing input is a question, a missing setting a message. Exceptions are written into the project memory first.
- The agent never enters endpoints or keys for a recipient and never relies on the assistant for its own work.

Scope

- **The look cannot be left** from the editor: fonts limited to the design-system stacks, colours to the NBG-inspired palette. The Tree tab's source editor is the one deliberate exception, for experts.
- The editor changes text, formatting, geometry, order, groups and shape style. Images, new elements and structure are the skill's job — regenerate.
- PowerPoint is an **export** of the HTML deck, never a second source to edit and re-import; newsletters, GPT and NotebookLM artifacts of the original project are excluded unless asked for.

SUMMARY

USING IT TOMORROW

One line for recipients, eight commands for builders, one marketplace to install from.

TELL THE RECIPIENT

Double-click any text to edit it (Enter applies, Esc cancels). Right-click for Resize / move shape (Shift+click or Shift+drag selects several; the toolbar orders, aligns, distributes and groups them), Edit SVG (the parts of a drawn icon, diagram or chart; one toolbar with Text, Shape and SVG tabs serves all three), Ask the assistant, Export to PDF and Save edited copy. After a skill update, `node deck.rebuild.mjs` refreshes the editing tools and the PDF.

WHERE IT LIVES

Public marketplace `github.com/BikS2013-coding-agents/nbg-design`: `/plugin marketplace add BikS2013-coding-agents/nbg-design`, then `/plugin install nbg-design@nbg-design`.

This deck: `github.com/BikS2013-presentations/deck-work-nbgdesign`, browsable on GitHub Pages and always current as `deck/NBG-Design-Skill-Explained-latest.html`; its generator and `rebuild.sh` live in the same repository.

THE EIGHT COMMANDS

```
S=<skill-root>/scripts
node $S/embed-assets.mjs deck.html  tokens → data URIs
node $S/add-deck-menu.mjs deck.html  inline the editor
node $S/verify-deck.mjs deck.html --strict  the gate
node $S/write-rebuild-script.mjs deck.html  deck.rebuild.mjs, delivered too
node $S/screenshot-deck.mjs deck.html -o shots  look
node $S/export-pdf.mjs deck.html  one page per slide
node $S/export-pptx.mjs deck.html  editable PowerPoint
node $S/compare-pptx.mjs deck.html deck.pptx  check it
```

NEXT STEPS

Ask for your next deck in this look and hand the three files to its readers.

Hand a deck to two colleagues, watch them edit it, and feed the friction back into the next release.

Ask for the PowerPoint version of a deck and read its comparison images.